

Real-Time Case Study: E-Commerce Order Analytics Using Java Streams

Scenario: You are working on a backend service for an E-Commerce platform. Order data is loaded into memory and management wants analytics reports using Java Streams API only. Students must solve all tasks using Stream operations instead of loops.

Domain Model Tasks

- 1 Create Order class with fields: orderId, customerName, category, amount, city, status
- 2 Create constructor, getters, and toString method
- 3 Create a List with at least 8 records

Basic Business Queries

- 1 Get all PAID orders
- 2 Count total orders
- 3 Find total revenue from PAID orders
- 4 Find highest value order
- 5 Find lowest value PAID order

Customer Insights

- 1 Get unique customer names
- 2 Find customers with orders above 50,000
- 3 Sort customers by order amount descending
- 4 Find top 3 highest orders

Category Analytics (GroupingBy)

- 1 Group orders by category
- 2 Count orders per category
- 3 Total sales per category
- 4 Maximum order per category
- 5 Average order value per category

City Analytics

- 1 Group orders by city
- 2 Find city with highest total revenue
- 3 Average order value per city

Partitioning Tasks

- 1 Partition orders into PAID vs NON-PAID
- 2 Count orders in each partition
- 3 Find max order amount in each partition

Advanced Stream Challenges

- 1 Find second highest order amount
- 2 Find duplicate order amounts
- 3 Create Map
- 4 Sort categories by total revenue
- 5 Check if any cancelled order is above 50,000

JUnit Testing Requirements

- 1 Write JUnit 5 test for total paid revenue
- 2 Test max order logic
- 3 Test grouping by category count
- 4 Test partition paid vs non-paid

Learning Outcomes

- 1 Use filter, map, reduce, sorted
- 2 Use groupingBy and partitioningBy
- 3 Work with Optional
- 4 Implement Top-N pattern
- 5 Write stream-based analytics